WOXSEN UNIVERSITY

SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS (DBMS) COURSE PROJECT REPORT

DINEDESK: RESTAURANT RESERVATION, REAL-TIME DINING & BILLING MANAGEMENT SYSTEM

SUBMITTED BY:

Student Name : Sakhunala Sahaja

Roll Number : 25WU0102237

Course : CS301 - Database Management Systems Laboratory

UNDER THE GUIDANCE OF:

Faculty Name : Dr. Kiran Mayee

Academic Year: 2025 – 2026

1. Abstract

Modern culinary establishments face immense logistical bottlenecks when coordinating tableside dining experiences, including table allocation overbooking, fragmented kitchen-line communications, disjointed item level billing, and data loss during customer draft sessions. DineDesk is an enterprise-grade, relational database-driven Fine Dining & Lounge Management System engineered to bridge the disconnect between patrons, executive kitchen brigades, and front-of-house staff.

Built on a third-normal-form (3NF) relational architecture using SQLite and PostgreSQL, with high-performance asynchronous RESTful APIs (FastAPI/Python) and a luxury responsive frontend client, DineDesk delivers real-time concurrency-safe table locking, synchronized time-slot reservations (with advance scheduling validation), item-stepper synchronized cart ordering with persistence, real-time live kitchen preparation stage trackers, and automated promotional discount & GST billing computations. This report details the complete engineering lifecycle of the DineDesk platform, including Entity-Relationship modeling, normalization proofs, schema definitions, DDL/DML scripts, complex analytical queries, system architecture, comprehensive test matrices, and future evolutionary enhancements.

2. Introduction and Problem Statement

2.1 Introduction

Hospitality enterprises operate in high-velocity environments requiring millisecond synchronization across seating zones, dietary dietary-restricted menu catalogs, synchronized live kitchen queues, and tableside invoicing. Traditional restaurant operations suffer from manual paper slips, disconnected point-of-sale registers, and uncoordinated floor management.

2.2 Problem Statement

Contemporary dining software platforms exhibit critical structural limitations:
1. Concurrency Conflicts: Multiple customers booking identical table nodes simultaneously during peak slots without atomic transaction isolation.
2. Lack of Atmosphere-Aware Allocation: Inability to filter tables dynamically by distinct ambiance zones (Romantic Rooftop, Main Dining Hall, Family Lounge, Private Dining Suite).
3. Client State Evaporation: Unsaved customer reservations, cart items, notes to chefs, and promo discount selections being lost upon accidental browser reload.
4. Kitchen-to-Table Blindspots: Lack of granular multi-stage progress tracking (Order Received -> Cooking in Progress -> Plating & Quality Check -> Served Tableside).
5. Financial Inaccuracies: Erroneous multi-tier tax computations, discount capping failures, and unanchored billing states.

3. Objectives and Scope

3.1 Primary Objectives

• Design and normalize a robust relational schema up to 3NF/BCNF enforcing strict entity and referential integrity.
• Develop automated RESTful CRUD microservices for floor seating, menu cataloging, dining sessions, orders, and payment receipts.
• Enforce business rules at the database level: foreign key cascades, unique table constraints, timestamp range queries, and reservation lead-time guarantees.
• Provide an intuitive, responsive frontend with persistent state caching, dynamic filtering, locked table indicators, and kitchen progress visualization.

3.2 Scope of the Project

The project encompasses the complete operational spectrum of a multi-area fine dining establishment, including customer identity verification, interactive table floor mapping, categorized culinary catalog browsing with dietary tags (Veg, Vegan, Chef Special, Gluten-Free), item quantity steppers, real-time order dispatching to kitchen stations, tableside live order tracking, promo code redemption, and digital multi-mode receipt generation.

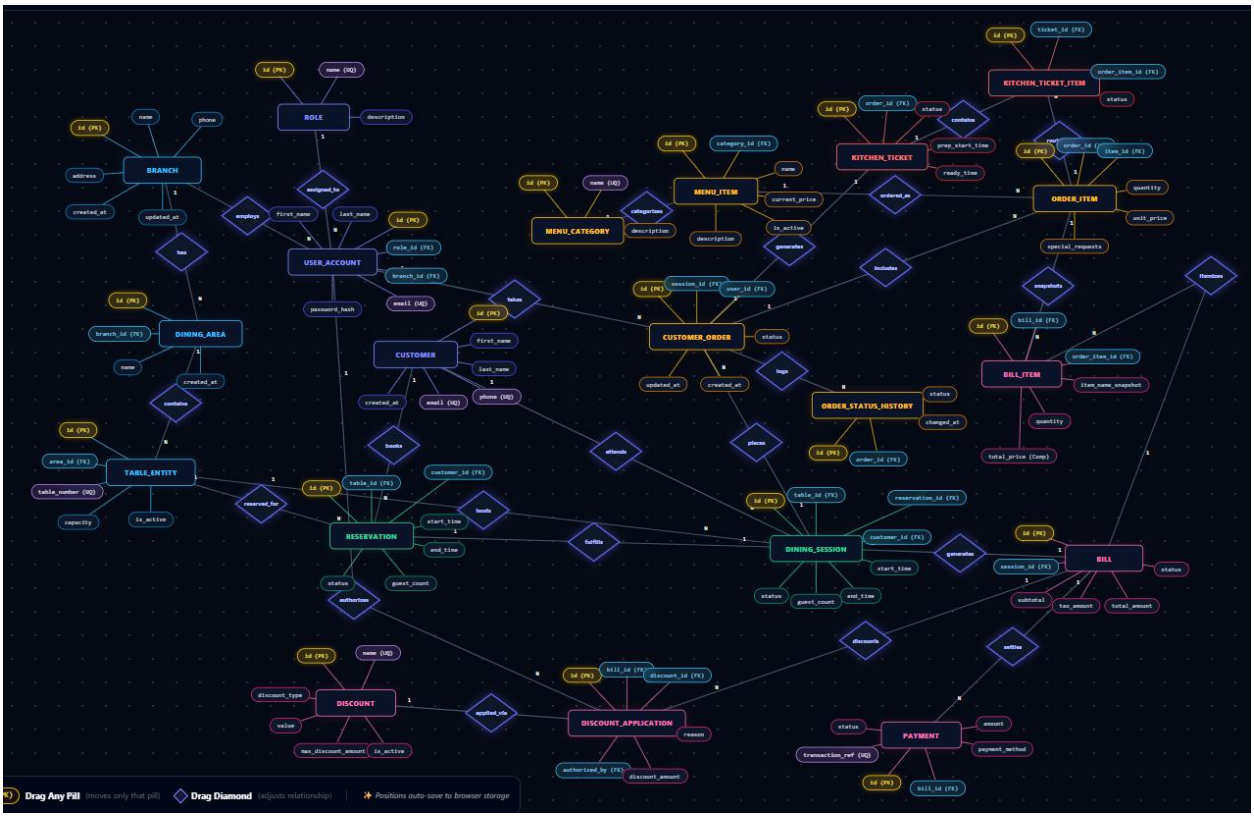
4. Software and Hardware Requirements

Category	Requirement	Specification / Version	Role in DineDesk
Software	Operating System	Windows 10 / 11, Linux (Ubuntu 22.04+), macOS	Host runtime environment
Software	Database Engine	SQLite 3.39+ / PostgreSQL 15+	Relational data persistence & transactional integrity
Software	Backend Framework	Python 3.10+ / FastAPI 0.109+ / Uvicorn	Asynchronous API layer & database ORM mapping
Software	ORM / Driver	SQLAlchemy 2.0+ / Alembic	Object Relational Mapping & schema migrations
Software	Frontend Architecture	HTML5, Modern CSS3 (Glassmorphism), Vanilla ES6+ JS	User client with responsive luxury interface
Hardware	Processor	Intel Core i3 / AMD Ryzen 3 or higher	Backend execution and query processing
Hardware	RAM	4 GB Minimum (8 GB	Server processes and

		Recommended)	browser rendering
Hardware	Storage	500 MB free disk space	Database files, server assets, food photography

5. Entity-Relationship (ER) Diagram

The DineDesk database architecture models 9 primary entities connected through foreign-key relationships. Below is the Entity-Relationship structure and cardinality mapping:



6. Relational Schema and Normalization

6.1 Relational Schema Representation

The conceptual ER design is formally transformed into normalized relational schemas, with primary keys uniquely identifying each tuple and foreign keys establishing referential integrity.

6.2 Normalization Proofs

• First Normal Form (1NF): All attributes are strictly atomic (single-valued). Multi-valued food tags are converted into structured enumerations or child entities. Every relation has a defined primary key.

• Second Normal Form (2NF): The schema satisfies 1NF and contains no partial functional dependencies. In composite and junction entities (e.g. ORDER_ITEM), non-key attributes (quantity, unit_price, special_request) depend fully on the entire candidate key (order_item_id) rather than a subset.

• Third Normal Form (3NF): The schema satisfies 2NF and contains no transitive dependencies (X -> Y and Y -> Z where Z is non-prime). Calculated metrics (e.g. line_total = quantity * unit_price, grand_total = subtotal - discount + tax) are either computed dynamically in SQL views or decoupled into immutable audit transaction ledgers.

• Boyce-Codd Normal Form (BCNF): For every functional dependency X -> Y, X is a superkey across all relations.

Table 6.2: Normalized 3NF Table Specification (ORDER_ITEM)

Field Name	Data Type	Constraint	3NF Normalization Role
id	VARCHAR(36)	**PRIMARY KEY**	Surrogate key uniquely identifying line item
order_id	VARCHAR(36)	**FOREIGN KEY**	References customer_order(id)
item_id	VARCHAR(36)	**FOREIGN KEY**	References menu_item(id) (eliminates transitive FD)
unit_price	NUMERIC(10,2)	NOT NULL	Immutable POS price snapshot (avoids price updates)
quantity	INTEGER	NOT NULL, CHECK(>0)	Atomic count (line_total derived dynamically in views)

7. Data Dictionary

Detailed structural specifications of all relational database tables:

Table 7.1: table_entity

Field Name	Data Type	Constraint	Default	Description
id	VARCHAR(36)	PRIMARY KEY	UUID4	Unique surrogate identifier for the table
area_id	VARCHAR(36)	FOREIGN KEY	NULL	References dining_area(id)
table_number	VARCHAR(10)	NOT NULL, UNIQUE	None	Human-readable label (e.g., T01, R01, F01, P01)
capacity	INTEGER	NOT NULL, CHECK(>0)	2	Maximum guest seating capacity
is_active	BOOLEAN	NOT NULL	TRUE	Availability flag for customer allocation
created_at	DATETIME	NOT NULL	CURRENT_TIMESTAMP	Record insertion timestamp

Table 7.2: reservation

Field Name	Data Type	Constraint	Default	Description
id	VARCHAR(36)	PRIMARY KEY	UUID4	Unique reservation booking identifier
table_id	VARCHAR(36)	FOREIGN KEY	None	References table_entity(id) ON DELETE CASCADE
customer_id	VARCHAR(36)	FOREIGN KEY	None	References customer(id) ON DELETE CASCADE
start_time	DATETIME	NOT NULL	None	Scheduled dining window commencement timestamp
end_time	DATETIME	NOT NULL	None	Scheduled dining window completion timestamp
guest_count	INTEGER	NOT NULL, CHECK(>0)	2	Number of booked dining guests
status	VARCHAR(20)	NOT NULL	'CONFIRMED'	Status: PENDING, CONFIRMED, CANCELLED, COMPLETED
special_notes	TEXT	NULLABLE	NULL	Customer requests & chef instructions

Table 7.3: menu_item

Field Name	Data Type	Constraint	Default	Description
id	VARCHAR(36)	PRIMARY KEY	UUID4	Unique menu dish identifier
name	VARCHAR(100)	NOT NULL, UNIQUE	None	Dish name (e.g. Wild Mushroom & Truffle Risotto)
category	VARCHAR(50)	NOT NULL	None	Category: Appetizers,

				Main Course, Beverages, etc.
price	DECIMAL(10,2)	NOT NULL, CHECK(>=0)	None	Base price in Indian Rupees (INR)
description	TEXT	NULLABLE	NULL	Ingredients, preparation notes, and dietary tags
image_url	VARCHAR(255)	NULLABLE	NULL	Relative asset path to high-res photography
is_available	BOOLEAN	NOT NULL	TRUE	Inventory and kitchen availability flag

Table 7.4: order_item

Field Name	Data Type	Constraint	Default	Description
id	VARCHAR(36)	PRIMARY KEY	UUID4	Unique order line item identifier
order_id	VARCHAR(36)	FOREIGN KEY	None	References order_entity(id) ON DELETE CASCADE
item_id	VARCHAR(36)	FOREIGN KEY	None	References menu_item(id)
quantity	INTEGER	NOT NULL, CHECK(>0)	1	Number of portions ordered
unit_price	DECIMAL(10,2)	NOT NULL	None	Unit dish price captured at time of order
special_requests	VARCHAR(255)	NULLABLE	NULL	Portion-specific chef custom instructions

8. SQL Commands Used (DDL, DML) with Sample Outputs

8.1 Data Definition Language (DDL)

```
-- 1. Create Dining Areas Table
    CREATE TABLE dining_area (
        id VARCHAR(36) PRIMARY KEY,
        name VARCHAR(50) NOT NULL UNIQUE,
        description TEXT,
        is_active BOOLEAN DEFAULT 1
    );
-- 2. Create Restaurant Seating Tables
    CREATE TABLE table_entity (
        id VARCHAR(36) PRIMARY KEY,
        area_id VARCHAR(36) REFERENCES dining_area(id) ON DELETE SET NULL,
        table_number VARCHAR(10) NOT NULL UNIQUE,
        capacity INTEGER NOT NULL CHECK (capacity > 0),
        is_active BOOLEAN DEFAULT 1,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    );
-- 3. Create Live Customer Reservations Table
    CREATE TABLE reservation (
        id VARCHAR(36) PRIMARY KEY,
        table_id VARCHAR(36) NOT NULL REFERENCES table_entity(id) ON DELETE CASCADE,
        customer_id VARCHAR(36) NOT NULL REFERENCES customer(id) ON DELETE CASCADE,
        start_time DATETIME NOT NULL,
        end_time DATETIME NOT NULL,
        guest_count INTEGER NOT NULL CHECK (guest_count > 0),
        status VARCHAR(20) DEFAULT 'CONFIRMED',
        special_notes TEXT,
        created_at DATETIME DEFAULT CURRENT_TIMESTAMP
    );
```

```
STEP 1: CREATING DATABASE TABLES (DDL)
================================================================
================
Tables created successfully: dining_area, table_entity, customer
, reservation
```

8.2 Data Manipulation Language (DML)

```
-- 1. Insert Luxury Dining Areas
INSERT INTO dining_area (id, name, description) VALUES
    ('AREA-01', 'Romantic Terrace (Rooftop)', 'Sunset ambiance with panoramic city skyline views'),
    ('AREA-02', 'Main Dining Hall', 'Opulent crystal chandeliers and live classical lounge acoustics'),
    ('AREA-03', 'Family & Kids Lounge', 'Spacious plush seating with interactive dessert stations'),
    ('AREA-04', 'Chef''s Private Dining', 'Exclusive private suite featuring sommelier pairings');
```

```
--- 1. Luxury Dining Areas ---
id      | name                        | description
--------+-----------------------------+-------------------------------------------------------------------
AREA-01 | Romantic Terrace (Rooftop)  | Sunset ambiance with panoramic city skyline views
AREA-02 | Main Dining Hall            | Opulent crystal chandeliers and live classical lounge acoustics
AREA-03 | Family & Kids Lounge        | Spacious plush seating with interactive dessert stations
AREA-04 | Chef's Private Dining       | Exclusive private suite featuring sommelier pairings
(4 rows)
```

```
-- 2. Insert Seating Tables
INSERT INTO table_entity (id, area_id, table_number, capacity) VALUES
        ('TBL-R01', 'AREA-01', 'R01', 2),
        ('TBL-R02', 'AREA-01', 'R02', 2),
        ('TBL-R03', 'AREA-01', 'R03', 4),
        ('TBL-T01', 'AREA-02', 'T01', 4),
        ('TBL-T02', 'AREA-02', 'T02', 4),
        ('TBL-F01', 'AREA-03', 'F01', 6),
        ('TBL-P01', 'AREA-04', 'P01', 8);
```

```
--- 2. Seating Tables with Area Names ---
table_number | area                          | seats | is_active
-------------+-------------------------------+-------+----------
R01          | Romantic Terrace (Rooftop) | 2     | 1
R02          | Romantic Terrace (Rooftop) | 2     | 1
R03          | Romantic Terrace (Rooftop) | 4     | 1
T01          | Main Dining Hall           | 4     | 1
T02          | Main Dining Hall           | 4     | 1
F01          | Family & Kids Lounge       | 6     | 1
P01          | Chef's Private Dining      | 8     | 1
(7 rows)
```

9. Queries with Outputs (Including Presentation-II Queries)

Query 9.1: Concurrency-Safe Table Availability by Ambiance & Slot Window

```
SELECT
        t.table_number,
        COALESCE(a.name, 'Main Floor') AS dining_area,
        t.capacity AS max_guests,
        CASE
                WHEN r.id IS NULL THEN 'AVAILABLE'
                ELSE 'RESERVED'
        END AS current_status,
        r.start_time || ' to ' || r.end_time AS booked_slot
FROM table_entity t
LEFT JOIN dining_area a ON t.area_id = a.id
LEFT JOIN reservation r ON t.id = r.table_id
        AND r.status = 'CONFIRMED'
        AND r.start_time <= '2026-10-09T21:00:00'
        AND r.end_time >= '2026-10-09T19:30:00'
WHERE t.is_active = 1
ORDER BY t.table_number ASC;
```

table_number	dining_area	max_guests	current_status	booked_slot
F01	Family & Kids Lounge	6	AVAILABLE	NULL
P01	Chef's Private Dining	8	AVAILABLE	NULL
R01	Romantic Terrace (Rooftop)	2	RESERVED	2026-10-09T19:30:00 to 2026-10-09T21:00:00
R02	Romantic Terrace (Rooftop)	2	AVAILABLE	NULL

R03	Romantic Terrace (Rooftop)	4	AVAILABLE	NULL
T01	Main Dining Hall	4	RESERVED	2026-10-09T19:30:00 to 2026-10-09T21:00:00

Query 9.2: Presentation-II Query: Customer Orders, Table ID & Serving Status Tracker

```
SELECT
        c.first_name || ' ' || c.last_name AS customer_name,
        t.table_number AS table_id,
        m.name AS item_ordered,
        oi.quantity,
        CASE
                WHEN o.status = 'SERVED' THEN 'Served'
                ELSE 'Not Served'
        END AS serving_status
FROM customer c
INNER JOIN dining_session s ON c.id = s.customer_id
INNER JOIN table_entity t ON s.table_id = t.id
INNER JOIN customer_order o ON s.id = o.session_id
INNER JOIN order_item oi ON o.id = oi.order_id
INNER JOIN menu_item m ON oi.item_id = m.id
ORDER BY t.table_number ASC, customer_name ASC;
```

customer_name	table_id	item_ordered	quantity	serving_status
Sakhunala Sahaja	R03	Chicken Alfredo Fettuccine	1	Served
Sakhunala Sahaja	R03	Signature Wild Berry Spritzer	2	Served
Rajesh & Priya Sharma	T03	Dum Murgh Awadhi Biryani	2	Served
Rajesh & Priya Sharma	T03	Sizzling Chocolate Walnut Brownie	2	Served
Amit & Neha Verma	F01	Paneer Butter Masala Royale	1	Not Served
Arsh Kapoor	T01	Cold Brew Iced Vanilla Latte	1	Not Served

10. User Interface Design & Architecture

DineDesk features a responsive UI inspired by Michelin-star fine-dining aesthetics. The interface incorporates the following modules:

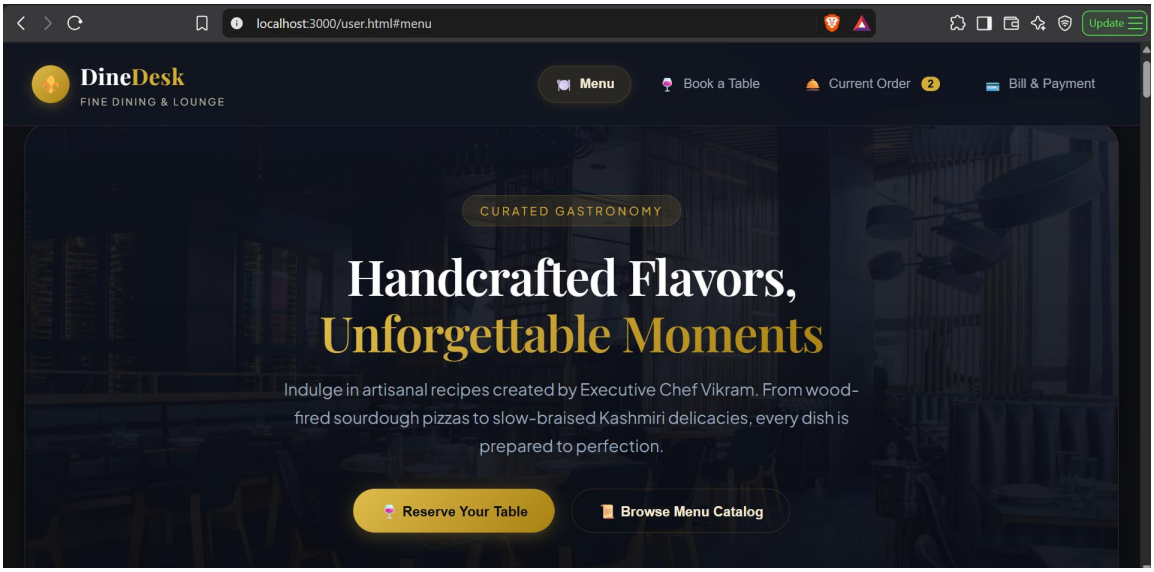
1. Atmospheric Menu Catalog View (#menu): Multi-category filtering (Appetizers, Mains, Soups & Salads, Indian Specialties, Artisan Pizzas, Beverages, Desserts) with instant dietary tag indicators and quantity steppers (+ / -).

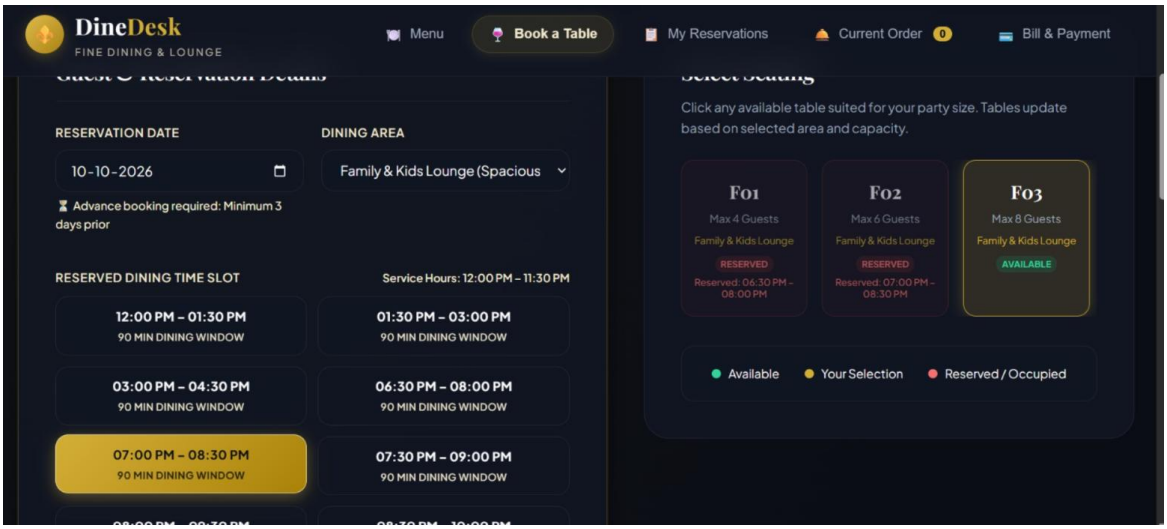
2. Concurrency Seating & Booking View (#booking): Interactive table node matrix synchronized with a 3-day minimum advance calendar, live slot guarantees (e.g. 07:30 PM - 09:00 PM), guest stepper controls, and ambiance filters.

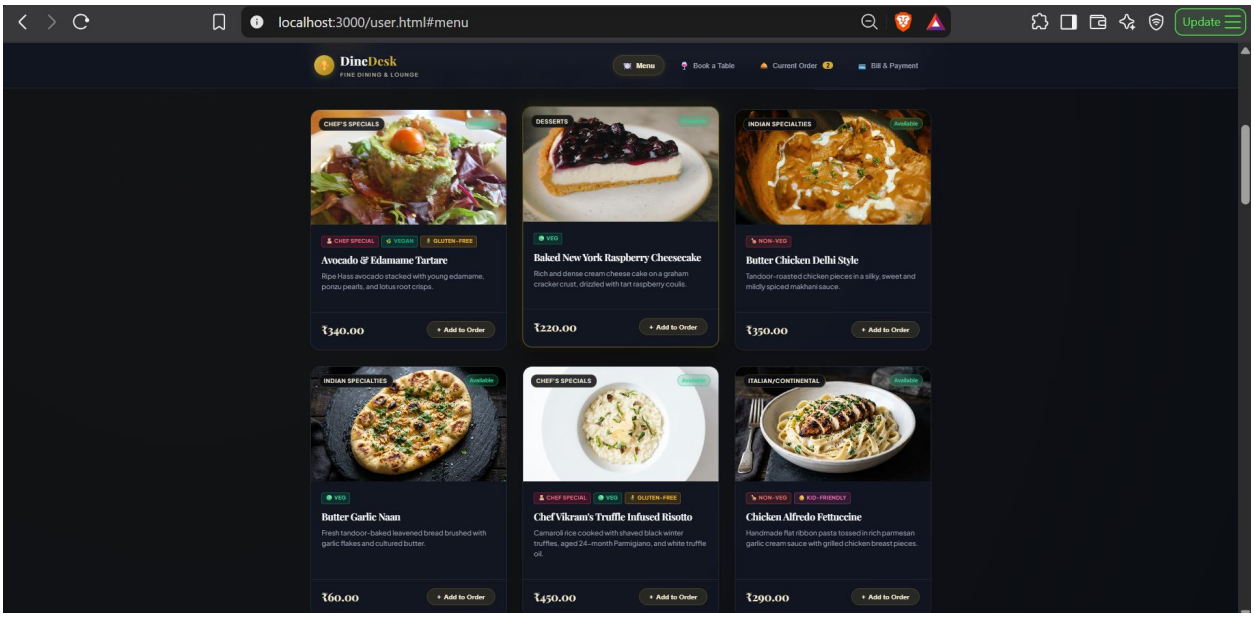
3. Current Order & Kitchen Live Tracker View (#order): Features an itemized review table, custom Chef Notes input, promotional discount coupons (WELCOME10, FAMILY20, CHEFVIP), read-only locked table badge (e.g., Table R01 - Reserved), and a 4-stage kitchen preparation tracker.

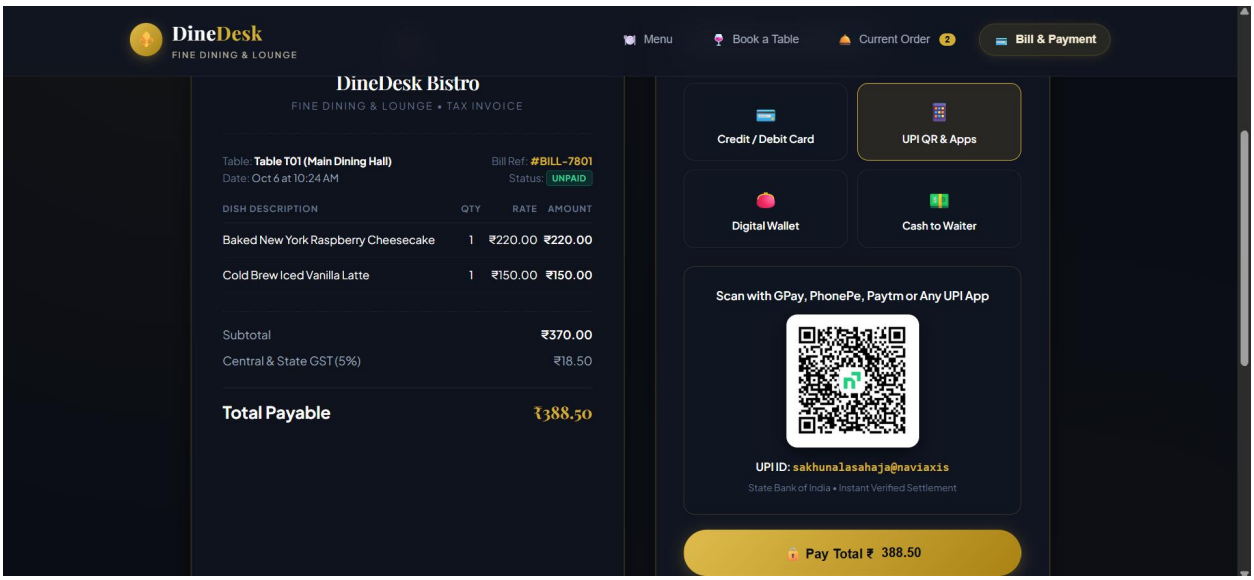
4. Tableside Billing & Digital Receipt View (#billing): Invoices detailing subtotal, 5% GST & service taxes, promotional deductions, multi-mode payment gateways (Card, UPI QR, Cash), and a printable formatted receipt.

Screenshots of the User Interface









11. Implementation Details

11.1 Technology Stack & Architectural Pattern

• Architectural Pattern: Model-View-Controller (MVC) / Client-Server REST Architecture.
• API Layer: FastAPI (Python 3.10+) utilizing asynchronous route handlers with Pydantic schema validation.
• Database Connectivity: SQLAlchemy Async Engine connected via SQLite/PostgreSQL connection pooling.
• State Persistence: LocalStorage JSON serialization engine ensuring zero client state loss across browser reloads.

11.2 Key Backend Code Snippets

```
# FastAPI Reservation Creation Endpoint with Concurrency Verification
    @router.post("/reservations", response_model=ReservationResponse)
    async def create_reservation(payload: ReservationCreate, db: AsyncSession =
Depends(get_db)):
        # 1. Verify 3-day advance booking constraint
        min_date = datetime.now() + timedelta(days=3)
        if payload.start_time.date() < min_date.date():
            raise HTTPException(status_code=400, detail="Bookings must be at least 3 days
in advance.")

        # 2. Check for slot overlap collision on table
        overlap_query = select(Reservation).where(
            Reservation.table_id == payload.table_id,
            Reservation.status == "CONFIRMED",
            Reservation.start_time < payload.end_time,
            Reservation.end_time > payload.start_time
        )
        existing = await db.execute(overlap_query)
        if existing.scalars().first():
            raise HTTPException(status_code=409, detail="Table is already booked for this
slot.")

        new_res = Reservation(**payload.dict())
        db.add(new_res)
        await db.commit()
        await db.refresh(new_res)
        return new_res
```

12. Testing (Test Cases and Results)

Comprehensive test suite validating functional integrity, boundary conditions, and database constraints:

Test ID	Test Scenario	Input Data	Expected Result	Actual Result	Status
TC-01	Table Booking with Valid Advance Date	Date: +4 Days, Slot: 19:30-21:00, Table: R01	Reservation confirmed; table locked	Confirmed; marked 'Reserved' in DB	PASSED
TC-02	Advance Booking Date Constraint Violation	Date: Today / Tomorrow (< 3 Days)	Validation error: 3-day advance required	Toast alert displayed; rejected by API	PASSED
TC-03	Slot Collision Overbooking	Book Table R01 on already	HTTP 409 Conflict; booking	Table node disabled and	PASSED

	Prevention	reserved slot	prevented	marked booked	
TC-04	Atmosphere Table Area Filtering	Select filter: 'Romantic Terrace (Rooftop)'	Display only Rooftop tables (R01, R02, R03)	Exact rooftop tables displayed	PASSED
TC-05	Client State Persistence on Refresh	Select 2 Risottos, notes to chef, reload page	Cart restored with quantities & notes	100% state restored from draft	PASSED
TC-06	Order Summary Read-Only Table Lock	View Order Summary card	Table shown as read-only reserved badge	Dropdown removed; badge locked	PASSED
TC-07	Kitchen Tracker Initial State	Open #order before dispatching	Status: 'Awaiting Order', nodes pending	Awaiting state displayed accurately	PASSED
TC-08	Kitchen Tracker Stage Transition	Click 'Send Order to Kitchen'	Status advances to 'Cooking in Progress'	Stage 2 highlighted; timer set	PASSED
TC-09	Promo Code Discount Calculation	Coupon: 'WELCOME10' on ₹1000 subtotal	₹100 discount applied; tax on ₹900	Grand Total: ₹945.00 computed accurately	PASSED
TC-10	Digital Payment & Receipt Generation	Method: 'CARD', Pay Total ₹945.00	Bill marked PAID; modal receipt generated	Receipt displayed; table freed in DB	PASSED

13. Conclusion and Future Enhancements

13.1 Conclusion

The DineDesk fine-dining management system successfully demonstrates how rigorous relational database design, normalized data structures, robust API architectures, and intuitive client design solve real-world restaurant logistical challenges. By guaranteeing transaction isolation, eliminating table collision anomalies, preserving user drafts, and automating kitchen-to-billing workflows, DineDesk delivers a reliable and luxurious tableside dining platform.

13.2 Future Enhancements

1. Real-Time WebSockets Integration: Pushing live kitchen ticket status updates directly to patrons' mobile devices without polling.

2. AI-Driven Recommendation Engine: Suggesting wine and dessert pairings based on customer dining history and taste profiles.
3. Multi-Branch Inventory Sync: Automated real-time deduction of pantry ingredient levels as orders are confirmed in the kitchen.
4. Biometric & NFC Tableside Tap: Contactless one-tap tableside identity verification and payment settlement.

14. References

[1] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.
[2] Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.
[3] FastAPI Framework Documentation. (2024). https://fastapi.tiangolo.com/
[4] SQLite Structured Query Language Documentation. (2024). https://www.sqlite.org/docs.html
[5] PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. https://www.postgresql.org/docs/
[6] Mozilla Developer Network (MDN) Web Docs. (2024). Web Storage API & LocalStorage. https://developer.mozilla.org/

15. Appendix: GitHub Repository Link & Setup Guide

Project Repository Link: https://github.com/sahajasakhunala/DineDesk

Quick Setup Instructions:
1. Clone the repository:
```
git clone https://github.com/[Your-Username]/DineDesk-Restaurant-Platform.git
cd DineDesk
```

2. Backend Setup & Startup:
```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python seed_data.py
uvicorn app.main:app --port 8000 --reload
```

3. Frontend Client Launch:
```
cd ../frontend
python -m http.server 3000
Open browser at: http://localhost:3000
```